COMPUTER ORGANIZATION & ASSEMBLY LANGUAGE (EE-2003)

ASSIGNMENT # 3

TOTAL MARKS = 130

- Assigned number is given to you in excel sheet provided with assignment consider it as decimal

FOR EXAMPLE: Assigned Number if your assigned number is **7823** where A0 is first digit of assigned number.

Fill in your roll numbers according to the specifications provided below.

	Assign Digit 0	Assign Digit 1	Assign Digit 2	Assign Digit 3
Short for Assigned Digit	**A0**	**A1**	**A2**	**A3**
Write Assigned Number Digit By Digit	**7**	**8**	**2**	**3**
	Assigned Byte 0		**Assigned Byte 1**	
Short for Assigned BYTE	**B0**		**B1**	
Byte in DECIMAL	**78**		**23**	
Convert byte to HEX B0H, B1H	**4E**		**17**	
Convert byte OCTAL B0Q, B1Q	**116**		**27**	
Convert byte BINARY B0B, B1B	**0100 1110**		**0001 0111**	
	Assigned WORD			
Short for Assigned WORD	**W**			
WORD in DECIMAL(W)	**7823**			
Convert WORD to HEX (WH)	**1E8F**			
Convert WORD OCTAL (WQ)	**17 217**			
Convert byte BINARY (WB)	**0001 1110 1000 1111**			
Assigned double WORD in HEXA (DD H)	**78237823H**			

EXAMPLE: Name is **HAMZA DAUD**

- *If your name starts with **MUHAMMAD** kindly use your second name*

	ZERO CHARACTER OF YOUR NAME	FIRST CHARACTER OF YOUR NAME	SECOND CHARACTER OF YOUR NAME	THIRD CHARACTER OF YOUR NAME	FOURTH CHARACTER OF YOUR NAME	FIFTH CHARACTER OF YOUR NAME	SIXTH CHARACTER OF YOUR NAME
Short for CHARACTER	**C0**	**C1**	**C2**	**C3**	**C4**	**C5**	**C6**
YOUR NAME CHARACTER BY CHARACTER	**H**	**A**	**M**	**Z**	**A**	**D**	**A**

1. Consider the following code and fill in the given registers and memory accordingly after each step? **(10 Marks)**

;**NOTE**

- **ADC** ax,bx ;AX+BX+CF(carry flag) ;ADC is add through carry
- You are supposed to run all 16 iterations
- Update **multiplicand,Result** and **multiplier** after every iteration

```
.data
    multiplicand dd 01E8Fh      ;Hexa of assigned word
    multiplier dw 005AAH
    result dd 0

.code
main PROC
mov eax,0

mov cl,16                       ;initialize bit count to 16
mov dx, multiplier              ;initialize bit mask

checkbit:
   shr dx,1
   jnc noadd                    ;skip addition if no carry

   mov ax, word ptr[multiplicand] ;mov LSW of multiplicand to ax
   add word ptr[result],ax        ;add LSW of multiplicand to result
   mov bx, word ptr[multiplicand+2];mov MSW of multiplicand to bx
   adc word ptr[result+2],bx  ;add MSW of multiplicand to result
noadd:
   shl word ptr[multiplicand],1    ;shift LSW multiplicand to left
   rcl word ptr[multiplicand+2],1
                                  ;rotate MSW of multiplicand to left
Loop checkbit
```

CF	BX	CF	AX	DX	CF		
	Multiplicand + 2		Multiplicand	Multiplier		Result+2	Result
0	0000000000000000	0	0011110100011110	0000001011010101	0	0000000000000000	0000000000000000
0	0000000000000000	0	0111101000111100	0000000101101010	1	0000000000000000	0011110100011110
0	0000000000000000	0	1111010001111000	0000000010110101	0	0000000000000000	0011110100011110
0	0000000000000001	1	1110100011110000	0000000001011010	1	0000000000000001	0011000110010110
0	0000000000000011	1	1101000111100000	0000000000101101	0	0000000000000001	0011000110010110

CF	BX	CF	AX	DX	CF		
	Multiplicand + 2		Multiplicand	Multiplier		Result+2	Result
0	0000000000000111	1	1010001111000000	0000000000010110	1	0000000000000101	0000001101110110
0	0000000000001111	1	0100011110000000	0000000000001011	0	0000000000000101	0000001101110110
0	0000000000011110	0	1000111100000000	0000000000000101	1	0000000000010100	0100101011110110

2. Modify and rewrite the above given code for following data declaration? **(5 Marks)**

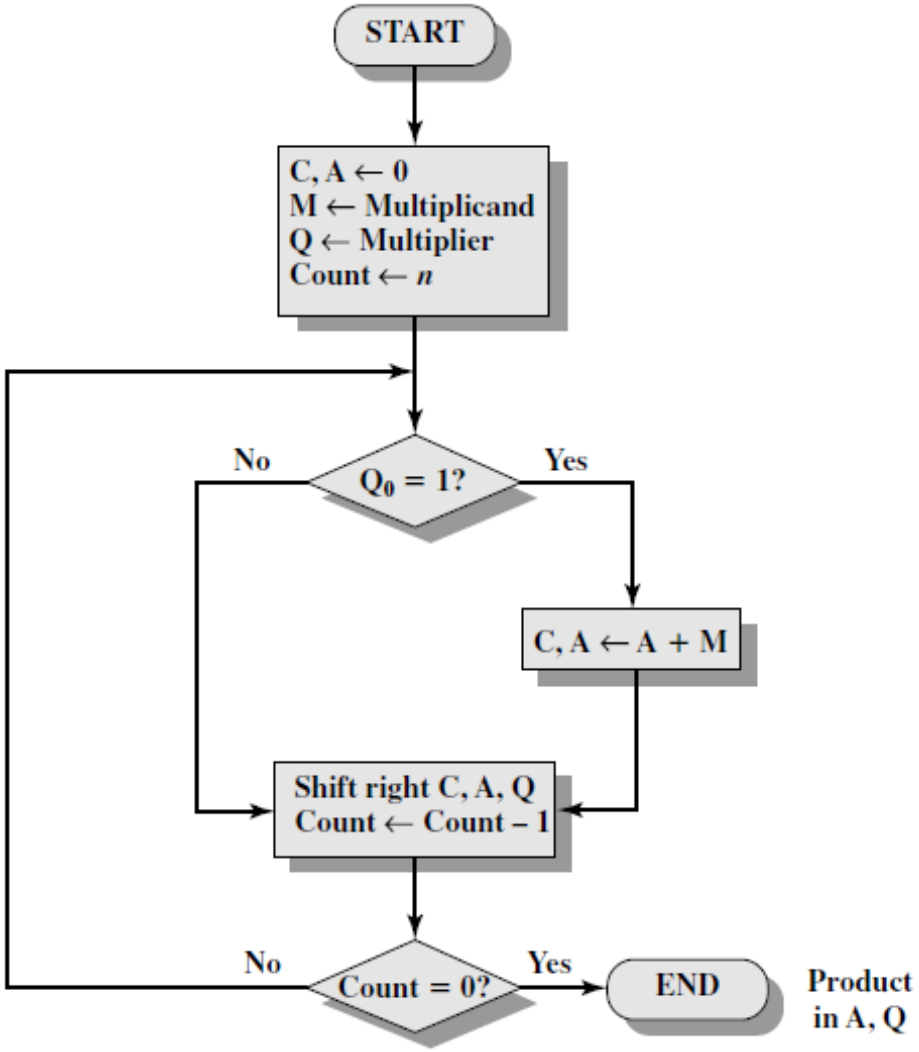
```
.data
  multiplicand DQ DH  ;value of Assigned doubleword in hexa
  multiplier DW WH    ;Assigned word in hexa
  result DQ 0         ;result of the multiplication
```

```
.data
 multiplicand DQ 078237823h
  multiplier DW 01E8FH
  result DQ 0
.code
  mov ax,0
  mov cl,16

  mov dx, multiplier
checkbit:
  shr dx,1
  jnc noadd
  mov ax, word ptr[multiplicand]
  add word ptr[result],ax
  mov ax, word ptr[multiplicand+2]
  adc word ptr[result+2], ax
  mov ax, word ptr [multiplicand+4]
  adc word ptr[result+4], ax
  mov ax, word ptr[multiplicand+6]
  adc word ptr[result+6],ax
noadd:
       shl word ptr[multiplicand],1        ;shift LSW multiplicand to left
       rcl word ptr[multiplicand+2],1
       rcl word ptr[multiplicand+4],1
     rcl word ptr[multiplicand+6],1        ;rotate MSW of multiplicand to
left
Loop checkbit
```

3. Perform unsigned binary multiplication using following given flow chart? **(10 Marks)**

NOTE: Your computer width is 8-bit, **Multiplicand** is $(01001110)_2$ and **Multiplier** is $(00010111)_2$



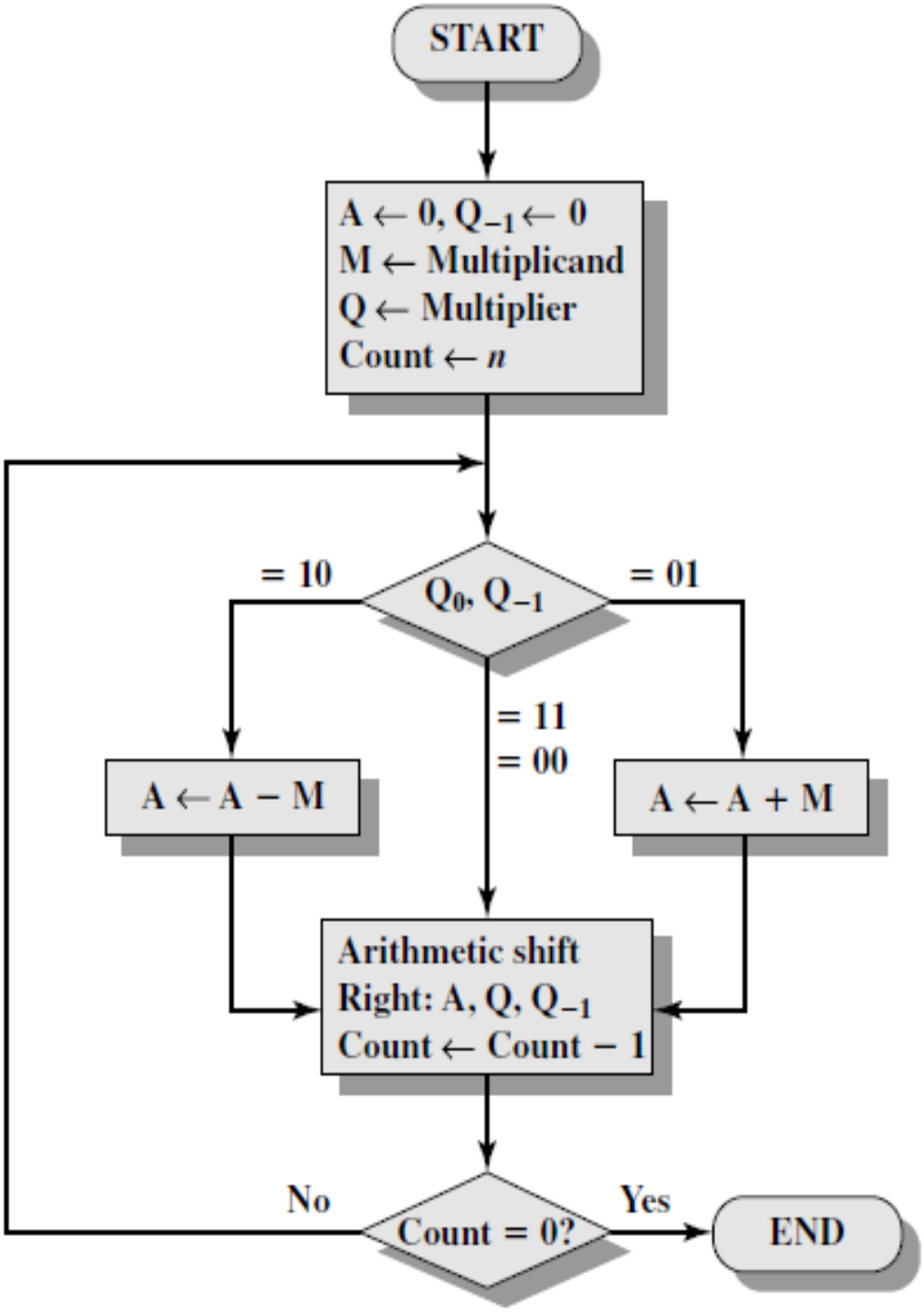
C	A (ACCUMULATOR)	Q (MULTIPLIER)	M (MULTIPLICAND)	
0	0 1 0 0 1 1 1 0	0 0 0 1 0 1 1 1	0 1 0 0 1 1 1 0	Add
0	0 0 1 0 0 1 1 1	0 0 0 0 1 0 1 1	0 1 0 0 1 1 1 0	Shift
C	A (ACCUMULATOR)	Q (MULTIPLIER)	M (MULTIPLICAND)	
0	0 1 1 1 0 1 0 1	0 0 0 0 1 0 1 1	0 1 0 0 1 1 1 0	Add
0	0 0 1 1 1 0 1 0	1 0 0 0 0 1 0 1	0 1 0 0 1 1 1 0	Shift
C	A (ACCUMULATOR)	Q (MULTIPLIER)	M (MULTIPLICAND)	
0	1 0 0 0 1 0 0 0	1 0 0 0 0 1 0 1	0 1 0 0 1 1 1 0	Add

	C	A (ACCUMULATOR)	Q (MULTIPLIER)	M (MULTIPLICAND)
	0	0 1 0 0 0 1 0 0	0 1 0 0 0 0 1 0	0 1 0 0 1 1 1 0
Shift				
	C	A (ACCUMULATOR)	Q (MULTIPLIER)	M (MULTIPLICAND)
Add				
0	0	0 1 0 0 0 1 0	0 0 1 0 0 0 0 1	0 1 0 0 1 1 1 0
	C	A (ACCUMULATOR)	Q (MULTIPLIER)	M (MULTIPLICAND)
Add	0	0 1 1 1 0 0 0 0	0 0 1 0 0 0 0 1	0 1 0 0 1 1 1 0
Shift	0	0 0 1 1 1 0 0 0	0 0 0 1 0 0 0 0	0 1 0 0 1 1 1 0
	C	A (ACCUMULATOR)	Q (MULTIPLIER)	M (MULTIPLICAND)
Add				
Shift	0	0 0 0 1 1 1 0 0	0 0 0 0 1 0 0 0	0 1 0 0 1 1 1 0
	C	A (ACCUMULATOR)	Q (MULTIPLIER)	M (MULTIPLICAND)
Add				
Shift	0	0 0 0 0 1 1 1 0	0 0 0 0 0 1 0 0	0 1 0 0 1 1 1 0
	C	A (ACCUMULATOR)	Q (MULTIPLIER)	M (MULTIPLICAND)
Add				
Shift	0	0 0 0 0 0 1 1 1	0 0 0 0 0 0 1 0	0 1 0 0 1 1 1 0

4.

4. Perform Multiplication using Booth's algorithm? **(10 Marks)**

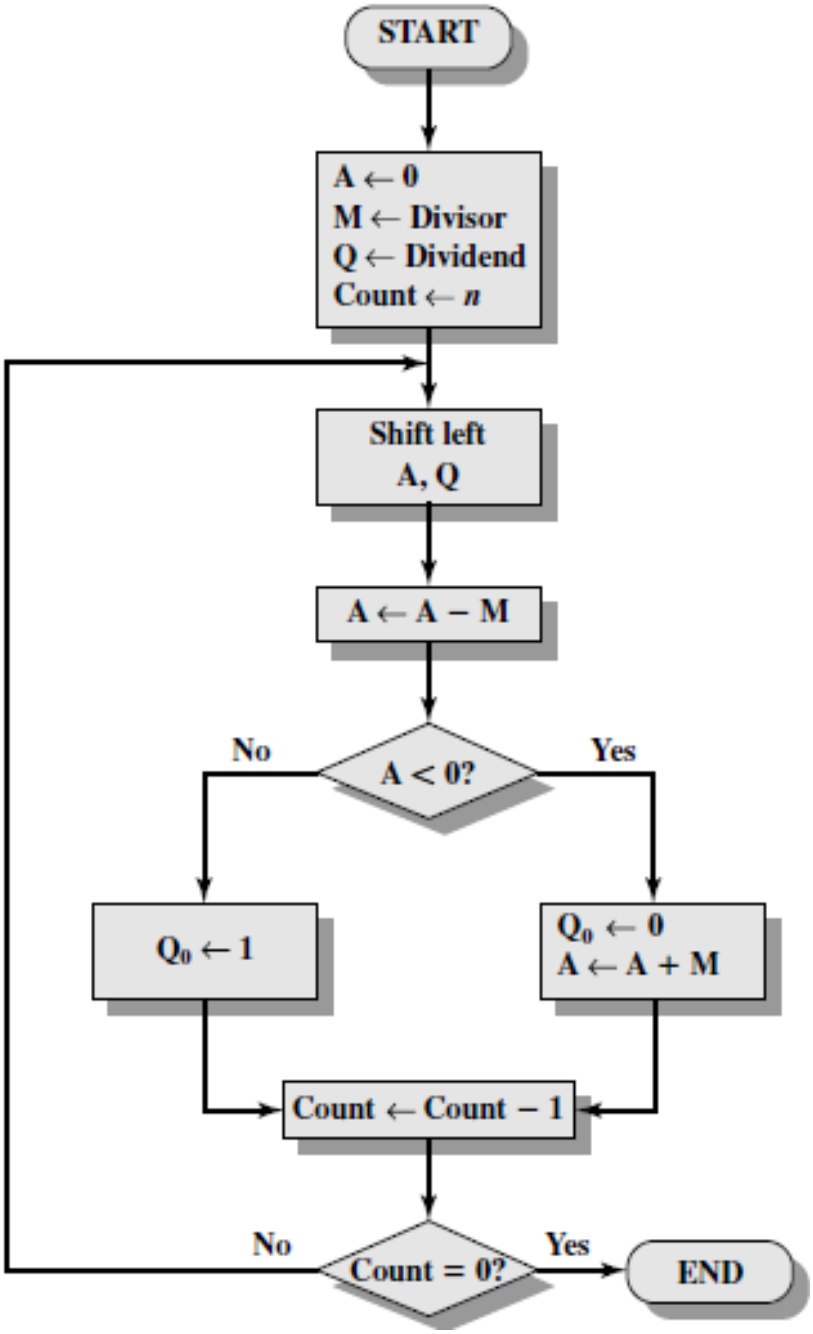
NOTE: Your computer width is 10-bit, **Multiplicand** is 2's complement of **B1H** assigned number (Means it's a signed and negative number), **Multiplier** is **0A5H?**



A	Q	Q_1	M	Operation	Iteration
0000000000	0010100101	0	1111101001	Initial	
0000010111	0010100101	0	1111101001	A-M	1
0000001011	1001010010	1	1111101001	Shift	
1111110100	1001010010	1	1111101001	A+M	2
1111111010	0100101001	0	1111101001	Shift	
0000010001	0100101001	0	1111101001	A-M	3
0000001000	1010010100	1	1111101001	Shift	
1111110011	1010010100	1	1111101001	A+M	4
1111111000	1101001010	0	1111101001	Shift	
1111111100	0110100101	0	1111101001	Shift	5
0000010011	0110100101	0	1111101001	A-M	6
0000001001	1011010010	1	1111101001	Shift	
1111110010	1011010010	1	1111101001	A+M	7
1111111001	0101101001	0	1111101001	Shift	
0000010000	0101101001	0	1111101001	A-M	8
0000001000	0010110100	1	1111101001	Shift	
1111110001	0010110100	1	1111101001	A+M	9
1111111000	1001011010	0	1111101001	Shift	
1111111100	0100101101	0	1111101001	Shift	10

5. Perform Unsigned binary division. Use flow chart to generate code as well **(10 Marks)**

NOTE: Your computer width is 8-bit, where dividend 17 **H** of assigned number, **Divisor** is **003H**



A/ACCUMULATOR	Q/DIVIDEND	COUNT	OPERATION
0000 0000	0001 0111		Initial
0000 0000	0010 1110	1	Shift
1111 1101	0010 1110		A-M
0000 0000	0010 1110		A+M
0000 0000	0101 1100	2	Shift
1111 1101	0101 1100		A-M
0000 0000	0101 1100		A+M
0000 0000	1011 1000	3	Shift
1111 1101	1011 1000		A-M
0000 0000	1011 1000		A+M
0000 0001	0111 0000	4	Shift
1111 1110	0111 0000		A-M
0000 0001	0111 0000		A+M
0000 0010	1110 0000	5	Shift
1111 1111	1110 0000		A-M
0000 0010	1110 0000		A+M
0000 0101	1100 0000	6	Shift
0000 0010	1100 0000		A-M
0000 0010	1100 0001		Q_0=1
0000 0101	1000 0010	7	Shift
0000 0010	1000 0010		A-M
0000 0010	1000 0011		Q_0=1
0000 0101	0000 0110	8	Shift
0000 0010	0000 0110		A-M
0000 0010	0000 0111		Q_0=1

6. Write a program that Left shift your character of your name three times to left and treat it as array. **Write final result in HEX.** **(6 Marks)**

```
nameSTRING db ‘C0’, ‘C1’, ‘C2’, ‘C3’, ‘C4’, ‘C5’, ‘C6’, ‘C7’, ‘C8’, ‘C9’
    ; C0 is first character of your name
    ; C9 is tenth character of your name
```

```
.data
            nameSTRING db 'R', 'O', 'H', 'A', 'I', 'L', 'G', 'U', 'L', 'B'
.code
    mov cl, 3
    Outerloop:
        mov bl, cl                      ;cl backup

        mov cl, 8
        Innerloop:
            shl nameSTRING[si+9], 1
            rcl nameSTRING[si+8], 1
            rcl nameSTRING[si+7], 1
            rcl nameSTRING[si+6], 1
            rcl nameSTRING[si+5], 1
            rcl nameSTRING[si+4], 1
            rcl nameSTRING[si+3], 1
            rcl nameSTRING[si+2], 1
            rcl nameSTRING[si+1], 1
            rcl nameSTRING[si], 1
        LOOP innerloop
        mov cl, bl                  ;cl restored
    LOOP outerloop
```

7. Apply the following piece of code and update the registers accordingly. **(4 Marks)**

```
mov al, 05fh

shr al, 1
```

REGISTER (AL)									CF
0	0	1	0	1	1	1	1		1

```
mov al, 0f5h

sar al, 1
```

REGISTER (AL)									CF
1	1	1	1	1	0	1	0		1

```
mov al, 05fh

rol al, 1
```

REGISTER (AL)									CF
1	0	1	1	1	1	1	0		0

```
mov al, 0f5h

rcl al,1
```

REGISTER (AL)									CF
1	1	1	0	1	0	1	0		1

8. You are working on the development of an encryption process for a secure communication protocol for a highly sensitive application. As part of the encryption process, the system needs to perform a series of bitwise operations on the provided data. Your task is to **design and implement an assembly program that can do these series of transformations on a 32-bit integer number mDATA.**

Suppose an 16-bit number **mFLAGS**, you have to transform the data by applying the series of following 2-bitwise operations on **mDATA** according to the specific bit value in **mFLAGS** as defined in the operation description provided below:
(10 Marks)

Operation #	Operation Description
1	**Double-Precision Shift Left:** Perform the **Double-Precision Shift Left**. The shift must be performed X times. The number X must be extracted from the **mFLAGS** using its last 4 bits (LSB). The destination must be **mDATA**, the source must be **mFLAGS**, and the count is X.
2	**Bitwise RIGHT Rotation:** Perform the **right bitwise** rotation. The rotations must be performed X times. The number X must be extracted from the **mFLAGS** using its first 5 bits. Ensure that the rotation is circular, meaning bits shifted out from one end are rotated back to the other end. Note: Not allowed to use any predefined rotate instruction like (ROL,ROR etc.)

Note: You have to generate 16-bit number **mFLAGS** using your **Roll Number** excluding the character and the first digit. For example if the number is **22i-9986** the mFLAGS must be **29986**.

Operation 1

```
.386
.model flat,stdcall
.stack 4096
ExitProcess proto,dwExitCode:dword

.data
mDATA dd 012345678h ;;dummy val 32 bit / 4 byte
mFLAGs dw 20977
mFLAGsc1 dw 20977

.code
main PROC

	and mFLAGS, 0fh ;; number of shifts (X - mask first 4 bits)
	mov cx, mFLAGS ;; move count to cl

	mov ebx, mDATA
	movzx eax, mFLAGsc1 ;; copy of mflag to use
	shld ebx, eax, cl ;; shld reg32,reg32,cl

INVOKE ExitProcess,0
```

```
main ENDP
END main
```

Operation 2

```
.386
.model flat,stdcall
.stack 4096
ExitProcess proto,dwExitCode:dword

.data
mDATA dd 012345678h ;;dummy val 32 bit / 4 byte
mFLAGs dw 20977
rotations dw ?

.code
main PROC

	and mFLAGS, 01fh ;; number of rotations (X - mask first 5 bits)
	mov ax, mFLAGS
	mov rotations, ax

	movzx ecx, rotations ;; apply rotations
	mov ebx, mDATA

	L1:
		shr ebx, 1
		jc Copy_Carry ;; if the rotations caused a carry

		Loop_Return:

		Loop L1
	jmp exit_1

	Copy_Carry:
		OR ebx, 80000000h ;; copying the carry to the MSB
		jmp Loop_Return

	exit_1:

INVOKE ExitProcess,0

main ENDP
END main
```

9. ASCII and Unpacked Decimal Arithmetic

A. You must dry-run the following program and fill in the registers after each iteration and execution.
Note: For DECIMAL_TWO your number will be equal to **(7.5 Marks)**
1 A_1 A_2 A_0 A_1 A_2A_3 A_0 A_1 A_2A_3 A_0 A_1 A_2A_3
For example 182 7823 7823 7823

```
.data
          decimal_one BYTE "985123456789765"
          decimal_two BYTE  Roll number as above
          sum BYTE (SIZEOF decimal_one + 1) DUP(0),0
.code
main Proc
mov esi,SIZEOF decimal_one - 1
mov edi,SIZEOF decimal_one
mov ecx,SIZEOF decimal_one
mov bh,0                                ; set carry value to zero
L1: mov ah,0                            ; clear AH before addition
mov al,decimal_one[esi]                 ; get the first digit
add al,bh                               ; add the previous carry
aaa                                          ; adjust the sum (AH =
carry)
mov bh,ah                               ; save the carry in carry1
or bh,30h                               ; convert it to ASCII
add al,decimal_two[esi]                 ; add the second digit
aaa                                     ; adjust the sum (AH =carry)
or bh,ah                                ; OR the carry with carry1
or bh,30h                               ; convert it to ASCII
or al,30h                               ; convert AL back to ASCII
mov sum[edi],al                         ; save it in the sum
dec esi                                 ; back up one digit
dec edi
loop L1
mov sum[edi],bh                         ; save last carry digit

mov bx,0
```

The following solution is in HEX notation.

Iteration 01:

AL (add al,decimal_two[esi])	**AL (After AAA)**
38	08

BH (or bh,30h)	**SUM (15-digits)**
30	00 00 00 00 00 00 00 00 00 00 00 00 00 00 38

Iteration 02:

AL (add al,decimal_two[esi])	**AL (After AAA)**
38	08

BH (or bh,30h)	**SUM (15-digits)**
30	00 00 00 00 00 00 00 00 00 00 00 00 00 38 38

Iteration 03:

AL (add al,decimal_two[esi])	**AL (After AAA)**
3F	05

BH (or bh,30h)	**SUM (15-digits)**
31	00 00 00 00 00 00 00 00 00 00 00 00 35 38 38

Iteration 04:

AL (add al,decimal_two[esi])	**AL (After AAA)**
37	07

BH (or bh,30h)	**SUM (15-digits)**
31	00 00 00 00 00 00 00 00 00 00 00 37 35 38 38

Iteration 05:

AL (add al,decimal_two[esi])	**AL (After AAA)**
3C	02

BH (or bh,30h)	**SUM (15-digits)**
31	00 00 00 00 00 00 00 00 00 00 32 37 35 38 38

Iteration 06:

AL (add al,decimal_two[esi])	**AL (After AAA)**
3A	00

BH (or bh,30h)	**SUM (15-digits)**
31	00 00 00 00 00 00 00 00 00 30 32 37 35 38 38

Iteration 07:

AL (add al,decimal_two[esi])	**AL (After AAA)**
3F	05

BH (or bh,30h)	**SUM (15-digits)**
31	00 00 00 00 00 00 00 00 35 30 32 37 35 38 38

Iteration 08:

AL (add al,decimal_two[esi])	**AL (After AAA)**
3D	03

BH (or bh,30h)	**SUM (15-digits)**
31	00 00 00 00 00 00 00 33 35 30 32 37 35 38 38

Iteration 09:

AL (add al,decimal_two[esi])	**AL (After AAA)**
38	08

BH (or bh,30h)	**SUM (15-digits)**
30	00 00 00 00 00 00 38 33 35 30 32 37 35 38 38

Iteration 10:

AL (add al,decimal_two[esi])	**AL (After AAA)**
35	05

BH (or bh,30h)	**SUM (15-digits)**
30	00 00 00 00 00 35 38 33 35 30 32 37 35 38 38

Iteration 11:

AL (add al,decimal_two[esi])	**AL (After AAA)**
3A	00

BH (or bh,30h)	**SUM (15-digits)**
31	00 00 00 00 30 35 38 33 35 30 32 37 35 38 38

Iteration 12:

AL (add al,decimal_two[esi])	**AL (After AAA)**
39	09

BH (or bh,30h)	**SUM (15-digits)**
30	00 00 00 39 30 35 38 33 35 30 32 37 35 38 38

Iteration 13:

AL (add al,decimal_two[esi])	**AL (After AAA)**
37	07

BH (or bh,30h)	**SUM (15-digits)**
30	00 00 37 39 30 35 38 33 35 30 32 37 35 38 38

Iteration 14:

AL (add al,decimal_two[esi])	**AL (After AAA)**
40	06

BH (or bh,30h)	**SUM (15-digits)**
31	00 36 37 39 30 35 38 33 35 30 32 37 35 38 38

Iteration 15:

AL (add al,decimal_two[esi])	**AL (After AAA)**
31	01

BH (or bh,30h)	**SUM (15-digits)**
31	31 36 37 39 30 35 38 33 35 30 32 37 35 38 38

Overflow detected, final SUM (16-Digit)
31 31 36 37 39 30 35 38 33 35 30 32 37 35 38 38

B. Convert the above code to Perform subtraction **DECIMAL_TWO** and **DECIMAL_ONE**.

Note: Use AAS instructions. **(7.5 Marks)**

```
.data
decimal_one BYTE "985123456789765"
decimal_two BYTE  Roll number as above
subtract BYTE (SIZEOF decimal_one+1) DUP(0),0
```

.code

```
    ; Start at the last digit position.
    mov esi,SIZEOF decimal_one - 1
    mov edi,SIZEOF decimal_one
    mov ecx,SIZEOF decimal_one
    mov bh,0                          ; set carry value (borrow) to zero

    L1:
        mov ah,0                      ; clear AH before subtraction
        mov al,decimal_one[esi]       ; get the first digit
        sub al,bh                      ; subtract any previous borrow
        mov bh,0                       ; reset borrow after applying it

        cmp al, decimal_two[esi]       ; compare if we need to borrow
        jb BorrowNeeded                ; jump if borrow is needed

        sub al, decimal_two[esi]       ; subtract the second digit
        jmp NoBorrow

    BorrowNeeded:
        add al, 10                     ; adjust for borrow
        sub al, decimal_two[esi]       ; subtract the second digit with value
        mov bh, 1                      ; set borrow for the next digit

    NoBorrow:
        or al, 30h                     ; convert AL to ASCII
        mov diff[edi], al              ; save the result in the diff array

        dec esi                        ; move to the next digit to the left
        dec edi
    LOOP L1

    mov diff[edi], bh                 ; save last borrow (if any) as a digit
```

10. Write a procedure that performs simple encryption by rotating each plaintext byte a varying number of positions in different directions. For example, in the following array that represents the encryption key, a **negative** value indicates a rotation to the **left** and a **positive** value indicates a rotation to the **right**. The integer in each position indicates the **magnitude** of the rotation:

key BYTE -5, 2, 3, 0, -1, -4, 3, -2, 4, 6

Your procedure should loop through a plaintext message and align the key to the first 10 bytes of the message. Rotate each plaintext byte by the amount indicated by its matching key array value. Then, align the key to the next 10 bytes of the message and repeat the process. Write a program that tests your encryption procedure. (**10 Marks)**

```
Include Irvine32.inc
.386
.model flat,stdcall
.stack 4096
ExitProcess proto,dwExitCode:dword

.data
plaintext BYTE "This is a test message for encryption...", 0 ; Null-terminated sample message
key BYTE -5, 2, 3, 0, -1, -4, 3, -2, 4, 6

.code
main PROC
    mov edx, OFFSET plaintext    ; Address of the plaintext message
    call EncryptMessage           ; Call encryption procedure

    mov edx, OFFSET plaintext   ; Display encrypted message
    call WriteString
    call Crlf

    exit
INVOKE ExitProcess,0
main ENDP


EncryptMessage PROC
    mov esi, OFFSET plaintext     ; ESI points to the plaintext
    mov edi, OFFSET key           ; EDI points to the key array
    mov ecx, LENGTHOF key         ; ECX stores length of the key array (10)

EncryptLoop:
    mov al, [esi]                 ; Load plaintext byte into AL
    cmp al, 0                     ; Check if we've reached the end of the message
    je EndEncrypt                 ; If null terminator, end the encryption process

    mov bl, [edi]                 ; Load corresponding key byte into BL
    test bl, bl                   ; Check if rotation is positive or negative
    jge RotateRight               ; If BL is non-negative, rotate right

    ; Rotate left for negative key values
    neg bl                        ; Convert BL to positive value for rotation count
    ;; save cl
    mov dl, cl
    mov cl, bl
    rol al, cl                    ; Rotate AL to the left by the magnitude in BL (cl)
    mov cl, dl

    jmp Continue
```

```
RotateRight:
    ;; save cl
    mov dl, cl
    mov cl, bl
    ror al, cl                  ; Rotate AL to the left by the magnitude in BL (cl)
    mov cl, dl

Continue:
    mov [esi], al               ; Store the rotated byte back into the message
    inc esi                     ; Move to the next byte in plaintext
    inc edi                     ; Move to the next byte in key

    mov eax, OFFSET key
    add eax, ecx
    cmp edi, eax                    ; Check if we reached end of the key array
    jne EncryptLoop             ; If not, continue encryption
    mov edi, OFFSET key         ; Reset key pointer to start of key array
    jmp EncryptLoop             ; Repeat encryption for next 10 bytes

EndEncrypt:
    ret
EncryptMessage ENDP

END main
```

11. Write a procedure BinarytoASC which will convert a binary integer into an ASCII binary string. The procedure must display the result. Write a program which tests your procedure as well. **(10 Marks)**

For example, a binary Integer 00100011 will be converted into a string (in ASCIIs) “00100011”.

Hint: Use SHL.

```
;-----------------------------------------------------
; BinToAsc PROC
;
; Converts a 32-bit binary integer in EAX to an ASCII binary
; representation and stores it in a buffer.
; Receives: EAX = binary integer to convert,
;           ESI = pointer to the buffer where ASCII binary digits will be stored
; Returns: The buffer pointed by ESI is filled with ASCII binary digits ('0' or
'1').
;-----------------------------------------------------
BinToAsc PROC
    push ecx                ; Save ECX on the stack
    push esi                ; Save ESI on the stack

    mov ecx, 32             ; Set loop counter for 32 bits

L1:
    shl eax, 1              ; Shift EAX left by 1, high bit goes into Carry flag
    mov BYTE PTR [esi], '0' ; Assume '0' as the default character
    jnc L2                  ; If no carry (high bit was 0), skip to L2
    mov BYTE PTR [esi], '1' ; If carry (high bit was 1), store '1' in buffer

L2:
    inc esi                 ; Move to the next buffer position
    loop L1                 ; Repeat until all 32 bits are processed

    pop esi                 ; Restore ESI from the stack
    pop ecx                 ; Restore ECX from the stack
    ret                     ; Return from the procedure
BinToAsc ENDP
```

12. Convert the following C++ program to assembly language using ***recursive stack procedures***. **(10 Marks)**

```cpp
// Function to calculate the nth Fibonacci number using recursion
int nthFibonacci(int n){

    // Base case: if n is 0 or 1, return n
    if (n <= 1){
        return n;
    }

    // Recursive case: sum of the two preceding Fibonacci numbers
    return nthFibonacci(n - 1) + nthFibonacci(n - 2);
}
```

```
.data
    n DWORD 2
    result DWORD ?

.code
main PROC
    mov eax, n              ; Load n into eax
    push eax                ; Push n onto the stack

    call nthFibonacci       ; Call the recursive Fibonacci function

    mov result, eax         ; Result returned in eax

main ENDP

nthFibonacci PROC
    ; Prologue
    push ebp                ; Save the base pointer
    mov ebp, esp            ; Set base pointer to stack pointer

    ; Get the value of n from the stack
    mov eax, [ebp+8]        ; Load the value of n (argument) into eax

    ; Base case: if n <= 1, return n
    cmp eax, 1              ; Compare n with 1
    jg RecursiveCase        ; If n > 1, jump to recursive case
    jmp Finish                 ; Else, return n (already in eax)

RecursiveCase:
    ; Recursive call: nthFibonacci(n - 1)
    dec eax                 ; Decrement eax (n - 1)
    push eax                ; Push n - 1 onto the stack
    call nthFibonacci       ; Call nthFibonacci(n - 1)
    add esp, 4              ; Clean up the stack (remove n - 1)
    push eax                ; Save the result of nthFibonacci(n - 1)

    ; Recursive call: nthFibonacci(n - 2)
    mov eax, [ebp+8]        ; Reload n
```

```
    sub eax, 2           ; Subtract 2 (n - 2)
    push eax             ; Push n - 2 onto the stack
    call nthFibonacci    ; Call nthFibonacci(n - 2)
    add esp, 4           ; Clean up the stack (remove n - 2)

    ; Combine results: nthFibonacci(n - 1) + nthFibonacci(n - 2)
    pop ecx              ; Get result of nthFibonacci(n - 1) from the stack
    add eax, ecx         ; Add nthFibonacci(n - 1) to nthFibonacci(n - 2)

Finish:
    ; Epilogue
    mov esp, ebp         ; Restore stack pointer
    pop ebp              ; Restore base pointer
    ret                  ; Return to the caller
nthFibonacci ENDP

END main
```

13. Convert the following C++ program to assembly language using ***recursive stack procedures.*** **(10 Marks)**

```
int findMaxRec(int A[], int n)
{
    // if n = 0 means whole array has been traversed
    if (n == 1)
        return A[0];
    return max(A[n-1], findMaxRec(A, n-1));
}
```

```
.data
	; Can be any values.
	array db 60, 4, 45, 6, -50, 0, 2
	array_size db sizeof array

	ans dw ?

.code
findMaxRec PROC
	push ebp
	mov ebp, esp

	cmp ecx, 0
		je CHECK_MAX

	dec ecx
	mov bl, BYTE PTR [esi+ecx]
	push ebx
	push ecx

	call findMaxRec

	CHECK_MAX:
		mov ebx, [ebp+12]
		cmp bl, al

		jge UPDATE
		jng RET_COND

		UPDATE:
			mov eax, ebx

		RET_COND:
			pop ebp
			ret 8
findMaxRec ENDP
```

```
main PROC
	mov esi, offset array
	movzx ecx, array_size

	mov eax, 1000
	mov ebx, 0
	add ecx, 1

	push esi
	push ecx

	call findMaxRec

	mov ans, ax
main ENDP
```

14. You are tasked to use **Sign Extension Instructions** for this question. **(10 Marks)**
For dividing signed numbers using the **IDIV** instructions, in the past, you have used movsx and movzx commands to extend the sign of the dividend in a larger register, as demonstrated:

```
.data
byteval sbyte -48
.code
movsx al, byteval
mov bl, 5
idiv bl
```

Write an Assembly Language code that uses **Sign Extension Instructions (CBW, CWD, and CDQ)** to perform the following divisions:

- **a)** Divide the signed byte -125 by 5
- **b)** Divide the signed word FFB1h by 7
- **c)** Divide the signed dword FFFFB0B1h by 6

Provide screenshots for each output (quotient and remainder).

Note: You **must** use CBW, CWD, and CDQ. Do **NOT** use movsx.

a)

```
.data
     byteVal SBYTE -48 ; D0 hexadecimal
.code
     mov al,byteVal  ; lower half of dividend
     cbw ; extend AL into AH
     mov bl,+5 ; divisor
     idiv bl  ; AL = -9, AH = -3
```

b)

```
.data
     wordVal SWORD -5000       ; Define a signed 16-bit word with value -5000

.code
     mov ax, wordVal           ; Move the signed value from wordVal into AX
     cwd                       ; Sign-extend AX into DX:AX (for division)

     mov bx, 256               ; Set BX to the divisor, 256

     idiv bx                   ; Perform signed division: DX:AX / BX
                              ; Quotient goes into AX, remainder goes into DX
     ; After idiv:
     ; AX = -19   (quotient)
     ; DX = -136  (remainder)
```

c)

```
.data
     dwordVal SDWORD +50000     ; Define a signed 32-bit double word w/ value
+50000

.code
     mov eax, dwordVal          ; Move the signed value from dwordVal into EAX
     cdq                        ; Sign-extend EAX into EDX:EAX (for division)

     mov ebx, -256              ; Set EBX to the divisor, -256

     idiv ebx                   ; Perform signed division: EDX:EAX / EBX
                               ; Quotient goes into EAX, remainder goes into EDX
     ; After idiv:
     ; EAX = -195  (quotient)
     ; EDX = +80   (remainder)
```